



Prompt Profissional para Correção de Autenticação e Isolamento de Dados

Aqui está um prompt estruturado e detalhado para orientar o Replit a corrigir os problemas de segurança e isolamento de dados:

PROMPT PARA REPLIT AGENT - CORREÇÃO CRÍTICA DE SEGURANÇA E ISOLAMENTO DE DADOS

Contexto do Problema

O sistema atual apresenta uma falha crítica de segurança: dados inseridos por um usuário estão sendo exibidos para todos os outros usuários autenticados, violando princípios fundamentais de privacidade e isolamento de dados. Especificamente, o módulo de Tarefas e potencialmente outros módulos compartilham informações entre usuários que não deveriam ter acesso.

Objetivo

Implementar isolamento completo de dados por usuário, garantindo que cada usuário visualize APENAS suas próprias informações, exceto em áreas explicitamente compartilhadas com permissões definidas.

Requisitos de Segurança Obrigatórios

1. Autenticação e Sessão

- Validar a sessão do usuário em TODAS as requisições ao backend
- Implementar verificação de token/sessão antes de qualquer operação CRUD
- Rejeitar requisições sem autenticação válida com status 401 (Unauthorized)
- Implementar timeout de sessão por inatividade

2. Isolamento de Dados (Data Isolation)

- Adicionar campo `user_id` ou `owner_id` em TODAS as tabelas de dados do usuário
- Implementar filtros automáticos em TODAS as queries de leitura: `WHERE user_id = current_user_id`
- Validar propriedade dos dados em operações de UPDATE e DELETE
- Implementar Row-Level Security (RLS) se usando PostgreSQL ou similar

3. Queries de Banco de Dados

Refatorar TODAS as queries para incluir filtro de usuário:

ERRADO:

```
SELECT * FROM tasks;
```

CORRETO:

```
SELECT * FROM tasks WHERE user_id = :current_user_id;
```

4. Áreas Compartilhadas (Shared Spaces)

Para recursos compartilhados, implementar:

- Tabela de permissões (shared_access ou permissions)
- Campos: resource_id, user_id, permission_level (read, write, admin)
- Query com JOIN para verificar acesso:

```
SELECT t.* FROM tasks t
LEFT JOIN shared_access sa ON t.id = sa.resource_id
WHERE t.user_id = :current_user_id
OR (sa.user_id = :current_user_id AND sa.permission_level IN ('read', 'write', 'admin'))
```

5. Middleware de Autorização

Criar middleware para:

- Extrair user_id do token/sessão autenticado
- Injetar automaticamente em todas as queries
- Validar permissões antes de executar operações
- Registrar logs de tentativas de acesso não autorizado

6. Endpoints da API

Revisar TODOS os endpoints para:

- Verificar autenticação no início da função
- Adicionar filtro de user_id em queries
- Retornar 403 (Forbidden) ao tentar acessar dados de outros usuários
- Validar entrada de dados para prevenir injection

7. Frontend - Boas Práticas

- Nunca confiar em filtros apenas no frontend
- Enviar token de autenticação em TODAS as requisições
- Implementar tratamento de erros 401/403
- Limpar dados de sessão ao fazer logout

Checklist de Implementação

Executar as seguintes etapas na ordem:

1. Auditoria de Código

- Identificar TODAS as queries de banco de dados
- Listar todos os endpoints que manipulam dados de usuários
- Mapear tabelas que precisam de campo `user_id`

2. Schema de Banco de Dados

- Adicionar coluna `user_id` (ou `owner_id`) em tabelas relevantes
- Criar índices em `user_id` para performance
- Criar tabela `shared_access` para recursos compartilhados
- Implementar constraints e foreign keys

3. Camada de Acesso a Dados

- Refatorar todas as queries para incluir filtro de usuário
- Criar funções auxiliares: `getUserData()`, `checkOwnership()`, `hasSharedAccess()`
- Implementar validação de propriedade antes de UPDATE/DELETE

4. Middleware e Autenticação

- Implementar middleware de autenticação global
- Extrair e validar `user_id` do token/sessão
- Adicionar middleware de autorização por rota

5. Testes de Segurança

- Testar com múltiplos usuários simultâneos
- Verificar que usuário A não vê dados do usuário B
- Testar permissões de áreas compartilhadas
- Validar tentativas de acesso não autorizado

6. Migração de Dados Existentes

- Associar dados órfãos ao usuário correto (se possível identificar)
- Definir estratégia para dados que não podem ser associados

Princípios de Segurança (OWASP)

- **Princípio do Menor Privilégio:** usuários acessam apenas o necessário
- **Defense in Depth:** múltiplas camadas de validação
- **Fail Secure:** em caso de erro, negar acesso por padrão
- **Never Trust User Input:** validar e sanitizar TODAS as entradas

Código de Exemplo - Middleware Express/Node.js

```
// Middleware de autenticação
const authenticateUser = (req, res, next) => {
  const token = req.headers.authorization;
  if (!token) return res.status(401).json({ error: 'Não autenticado' });

  try {
    const decoded = verifyToken(token);
    req.userId = decoded.userId; // Injeta userId na requisição
    next();
  } catch (error) {
    return res.status(401).json({ error: 'Token inválido' });
  }
};

// Exemplo de query segura
app.get('/api/tasks', authenticateUser, async (req, res) => {
  const tasks = await db.query(
    'SELECT * FROM tasks WHERE user_id = $1 OR id IN (SELECT resource_id FROM shared_acc'
    [req.userId]
  );
  res.json(tasks);
});
```

Prioridade

▮ **CRÍTICO:** Esta correção deve ser tratada como PRIORIDADE MÁXIMA, pois envolve vazamento de dados sensíveis entre usuários.

Documentação Necessária

Após implementação, documentar:

- Modelo de permissões implementado
- Como adicionar novos recursos com isolamento
- Processo para criar áreas compartilhadas
- Logs de auditoria de acesso

Por favor, implemente estas correções seguindo as melhores práticas de segurança (OWASP Top 10) e governança de dados (LGPD/GDPR compliance).

Este prompt fornece diretrizes técnicas específicas e acionáveis para o Replit corrigir os problemas de segurança de forma profissional e abrangente. Você pode ajustá-lo conforme a stack tecnológica específica da sua aplicação (framework, banco de dados, etc.).

Gostaria que eu adaptasse o prompt para alguma tecnologia específica que você está usando?